

Iteracyjny Algorytm Euklidesa: Optymalny Wybór dla Obliczania NWD

Dlaczego Iteracja Zwycięża Rekurencję w Algorytmie Euklidesa?

Wybierając między iteracyjną a rekurencyjną wersją algorytmu Euklidesa do obliczania NWD, profesjonaliści niemal zawsze stawiają na **podejście iteracyjne**, i to z ważnych powodów. Chociaż wersja rekurencyjna zachwyca swoją elegancją i bliskością do matematycznego zapisu, jej iteracyjny odpowiednik wygrywa w kluczowych kategoriach: **wydajności i stabilności** ⚙️. Implementacja z pętlą **while** jest znacznie lżejsza dla zasobów komputera – nie tworzy ona kolejnych wywołań funkcji na stosie, co eliminuje ryzyko jego przepełnienia (*stack overflow*) przy pracy z dużymi liczbami. Mniejsze zużycie pamięci i szybsze działanie, pozbawione narzutu związanego z rekurencją, czynią z wersji iteracyjnej bezpieczniejszy i bardziej niezawodny wybór do każdego poważnego projektu programistycznego. To pragmatyczne rozwiązanie, które gwarantuje, że kod będzie działał solidnie, niezależnie od skali problemu.

Iteracyjny Algorytm Euklidesa w Praktyce (Przykład Kodu C++)

```
#include <iostream>

// Funkcja do obliczania NWD metodą iteracyjną
int nwd_iteracyjnie(int a, int b) {
    while (b != 0) {
        int r = a % b; // Oblicz resztę z dzielenia
        a = b;         // 'b' staje się nowym 'a'
        b = r;         // reszta 'r' staje się nowym 'b'
    }
    // Gdy b == 0, wynikiem jest a
    return a;
}

int main() {
    int liczba1 = 48;
```

```

int liczba2 = 18;
int wynik = nwd_iteracyjnie(liczba1, liczba2);

std::cout << "NWD(" << liczba1 << ", " << liczba2 << ") = " << wynik << std::endl;

return 0;
}

```



```

#include <iostream>

// Funkcja do obliczania NWD metodą iteracyjną
int nwd_iteracyjnie(int a, int b) {
    while (b != 0) {
        int r = a % b; // Oblicz resztę z dzielenia
        a = b;         // 'b' staje się nowym 'a'
        b = r;         // reszta 'r' staje się nowym 'b'
    }
    // Gdy b == 0, wynikiem jest a
    return a;
}

int main() {
    int liczba1 = 48;
    int liczba2 = 18;
    int wynik = nwd_iteracyjnie(liczba1, liczba2);

    std::cout << "NWD(" << liczba1 << ", " << liczba2 << ") = " << wynik << std::endl;

    return 0;
}

```

Schemat blokowy:

